

First Order Predicate Logic

MATH230

Te Kura Pāngarau
Te Whare Wānanga o Waitaha

- ➊ Motivation
- ➋ Language and Grammar
- ➌ First Order Rules of Inference

Recall that we have introduced logic so that we can write mathematics in a precise enough manner to analyse proof and provability.

In defining propositional logic we suppressed a lot of detail into a single propositional variable P . It's time now to look at each proposition in more detail and parse out different components. *Like physicists finding parts (particles) within their “atoms”.*

In this lecture we will approach the topic of first-order logic by determining which components are still missing from our language.

Fundamental Theorem of Arithmetic

For all integers $n > 1$ either n is a prime or n is a finite product of primes.

Details of Propositions

In the statement of the fundamental theorem of arithmetic was found the following details within' propositions:

T	:	"Universe of discourse" or "Type" or "Set".
$n : T$	:	Terms of that type.
is prime	:	"Predicate" or "Property" which makes propositions.
For all	:	Quantification over the Type.

If we consider "is prime" or " $n > 1$ ", then we find even more structure. Each of these are making "existential" claims; that is, claims about whether or not certain other terms exist. Previously we put all of these details into one propositional variable. First Order Predicate Logic brings this information out into the open, hence allowing for more nuanced propositions.

Terms and their Properties

Mathematicians and computer scientists speak about terms of specific types:

$3 : \mathbb{N}$	$-4 : \mathbb{Z}$	$\pi : \mathbb{R}$
$\text{True} : \text{Bool}$	$[1,2,3] : \text{List } \mathbb{N}$	

Furthermore they are interested in proving that these terms have certain properties:

$\text{even} : \mathbb{N} \rightarrow \text{Prop}$	$2 \mapsto \text{even } 2$
$\text{prime} : \mathbb{N} \rightarrow \text{Prop}$	$4 \mapsto \text{prime } 4$
$\text{null} : \text{List } \mathbb{N} \rightarrow \text{Prop}$	$[1,2,3] \mapsto \text{null } [1,2,3]$
$\text{equal} : \mathbb{N} \times \mathbb{N} \rightarrow \text{Prop}$	$(a, b) \mapsto a = b$

These properties are called predicates. In first order predicate logic they are thought of as functions that return propositions.

Variables: $x, y, z \dots$

Some of these statements are written *generally*. That is to say, the values associated to the particular letters are left open to be anything within' the domain of discourse:

- If n is an integer...
- If a, b are integers...
- If p is some prime...
- There exists an integer $n \dots$

So our language will need the ability to express *variables* which can be interpreted in some specific *type*. We will also refer to a general type and general predicates on types; thus we will use variables for terms, types, and predicates. Lowercase variables will be used for terms, whereas uppercase variables will be used to denote types and predicates.

Quantification: $\forall$ and $\exists$

Quantification goes hand-in-hand with this expression of generality and use of variables. What we mean when we say:

If n is an integer, then $\text{even}(n) \vee \text{odd}(n)$

Every integer satisfies the proposition $\text{even}(n) \vee \text{odd}(n)$.

Similarly claims like *there exists* an n such that $\phi(n)$, are quantifying over some type. Of all the terms in that type, at least one of them satisfies the predicate $\phi(n)$.

Functions: $f, g, h, \dots$

Sums, products, and exponentiation were used to state the above claims. These are examples of the fundamental notion of *function*. Mathematicians and computer scientists use functions to express all sorts of things: we would be lost with them.

The idea being that functions take in some number of inputs (from the domain) and return one element of the domain of discourse.

Note: this makes them different from predicates/relations as predicates do not return elements of the domain. Rather, they return propositions.

PL to First-Order Logic

In summary, we need to add the following to propositional logic:

- Domain type
- Constants
- Functions
- Variables
- Properties/Relations (Predicates)
- Quantifiers

These, with propositional logic, form the language of *first-order predicate logic*.

Alphabet of FoL

We use the following standard language of *first-order predicate logic* to build terms, well-formed formulae, and propositions.

- Type variables $P, Q, R, S, T, \dots$
- Term variables $x, y, z, \dots$ or $x_1, x_2, x_3, \dots$
- Constants $a, b, c, \dots$ or $c_1, c_2, c_3, \dots$
- Predicate (relation) symbols $R_0, R_1, R_2, \dots$
- Function symbols $f_0, f_1, f_2, \dots$
- Logical connectives from propositional logic $\neg, \wedge, \vee, \rightarrow, \perp$
- Quantifiers $\forall$ and $\exists$

Note: For clarity we also use parentheses and commas when writing expressions. These can be included in the symbols of our language as well.

Universal and Existential Quantifiers

The symbol $\forall$ will be interpreted as “for all...”

The symbol $\exists$ will be interpreted as “there exists...”

Example: For all real numbers $\epsilon > 0$ there exists another real number $\delta > 0$

For us, in these notes, quantifiers will always be stated with a domain of quantification; so called bounded quantification. This departs from most of the literature on logic which considers unbounded quantification.

Terms of a domain T are defined inductively as follows:

- Variables and constants are terms
- If f is an n -ary function and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term.

Well-Formed Formulae (WFF) are defined inductively as follows:

- $\perp$ is a WFF.
- An n -ary predicate applied to n -terms $R(t_1, \dots, t_n)$ is a WFF.
- If α, β are WFF, then $\neg\alpha$, $(\alpha \vee \beta)$, $(\alpha \wedge \beta)$, and $(\alpha \rightarrow \beta)$ are WFF.
- If α is a WFF and x is a variable, then $(\forall x : T, \alpha)$ and $(\exists x : T, \alpha)$ are WFF.

These are the syntactic rules for building correct strings of symbols.

Let T be a type, R be a two-place (binary) predicate, P a one-place (unary) predicate, f a binary function, and a, b be constants. Determine which of the following are WFF and which are not.

① Pa

② a

③ Pab

④ $\exists x : T, Rxb \rightarrow Px$

⑤ $\exists x(Px \vee Rbx)$

⑥ $\forall x : T, (\exists y : T, (R(x, y)))$

⑦ $\exists x : T, (\forall y : T, (R(x, y)P(y)P(x)))$

⑧ $f(y, b)$

Binding Conventions

If you want to drop parentheses, then your expressions will be interpreted using the following binding conventions:

- $\forall, \exists, \neg$ bind most tightly;
- $\wedge$ and $\vee$ bind more tightly than $\rightarrow$;
- $\rightarrow$ binds more tightly than $\leftrightarrow$.

With this convention one can unambiguously interpret the string:

$$\neg \exists x : T, Rx \rightarrow \neg Q \wedge \forall x : T, \neg Px$$

as the following wff:

The last few slides have said how we can build strings that are part of the language of first order logic. So far, these are just strings of symbols that fit together into terms and wff with the grammar defined.

They have no meaning, *yet*. We don't work this abstractly. Instead we choose to construct specific base types with constants, functions, and predicates to express ideas and data precisely. It's this choice of “signature” that determines the meaning of the strings in this language.

First Order Languages

In practice we decide on a few constants, predicates, and functions depending on the part of mathematics (or computer science) we want to study. Thus specifying a *first order language* by defining the *signature* of the language.

Example

Let us say we have the constant 0 , unary function s , binary functions $+$ and $\times$, and $=$ as the only relation.

We collect these together into a first order language, which we can denote $\mathcal{L}$, with signature $\{0, s, +, \times, =\}$

WFF in a First Order Language

Given a first order language $\mathcal{L}$ with signature $\{0, s, +, \times, =\}$ we see that the constants and function symbols generate the base type of the first order language:

Prop in a First Order Language

Given a first order language $\mathcal{L}$ with signature $\{0, s, +, \times, =\}$ whose terms generate type $\mathbb{N}$ we may write the following Propositions:

- $\forall x : \mathbb{N}, \forall y : \mathbb{N}, (= (x, y) \vee \neg (= (x, y)))$
- $\exists x : \mathbb{N}, (\forall y : \mathbb{N}, \neg (= (x, s(y))))$
- $= (+ (0, s(0)), 0)$

Prefix, Infix, Postfix

In the textbook Logic and Proof¹ they pick the following first order language for the entire chapter on first order logic:

$$\mathcal{L} : \{1, 2, 3, \textit{add}, \textit{mul}, \textit{square}, \textit{even}, \textit{odd}, \textit{prime}, \textit{lt}, =\}$$

- Every natural number is even or odd, but not both.
- $\textit{Even}(n) \leftrightarrow \exists x : \mathbb{N}^+, (n = \textit{mul}(2, x))$.
- If x is even, then x^2 is even.
- There exists a prime number that is even.

¹Note: this uses an outdated version of Lean, but is still a good resource for the theory of this course.

$\mathcal{L} : \{1, 2, 3, \text{add}, \text{mul}, \text{square}, \text{even}, \text{odd}, \text{prime}, \text{lt}, =\}$

$$\mathcal{L}_{\mathbb{B}} : \{\text{True}, \text{False}, \wedge, \vee, \neg, \rightarrow, =\}$$

Arguments and Proofs

Recall that our intention is to understand when an argument written in our formal language is valid; when one wff is said to follow from a set of other wff.

To this end we now write down rules of inference for the new structure of first-order predicate logic. Namely, introduction and elimination rules for the quantifiers.

Bounded Quantification

Thus far we have considered the terms generated by the signature of a first-order language and collecting them together into a type. We have taken the quantifiers to range over this type.

This explicit mention of the type appears atypical of presentations of first-order logic. **For better, or for worse, we will present the rules of inference without explicit reference to a type of terms.**

We turn, again, to the BHK for some idea of what it means to introduce/eliminate quantifiers:

- $\forall$ a proof of $\forall x P(x)$ is an algorithm that, applied to *any* object t returns a proof of $P(t)$.
- $\exists$ to prove $\exists x P(x)$ one must construct an object t and a proof of $P(t)$.

This presentation is taken from the Stanford Encyclopedia of Philosophy article by Bridges, Palmgren, and Ishihara.

Universal Quantification

Suppose we had, in the course of a proof, deduced the statement $\forall x \alpha$, where α is a well-formed formula that (may) contain the variable x . What can we conclude from that?

$$\frac{\begin{array}{c} \Sigma \\ \mathcal{D} \\ \forall x \alpha[x] \end{array}}{?} \quad \forall E$$

Following the BHK interpretation of $\forall$ we should then be able to *substitute* any term, t , in place of each instance of the variable x in the well formed formula α .

Substitution Rules

If we don't take care when substituting for variables, then we can go astray.

Definition

If a variable, x , is used to quantify over α — as in $\gamma = \forall x \alpha$ — then we say any instance of x in α is bound in γ by the quantifier. Similarly for the existential quantifier.

Definition

If a variable, x , is not bound in γ , then we say that it is free in γ .

Bound and free variables act differently when interpreted; they play different roles. They distinguish between the propositions we want to prove and the merely syntactically correct formulae used to build those propositions.

Free vs. Bound Variables

We can translate the first-order expression: $\forall x(x < y)$ as “every x is less than ____”. The interpretation of this statement depends on the interpretation of y ; because y is free. It’s truth will depend on the interpretation of y .

On the other hand the interpretation of $\exists y(\forall x(x < y))$ doesn’t depend on y . As there either exists something that satisfies this sentence, or there doesn’t.

For this reason we should not make free variables bound, or vice versa, when substituting.

Substitution Notation

If α is a well-formed formula in which x is a free variable and y is free to be substituted in for x , then we denote

$$\alpha[x/y] \quad \alpha \text{ with } \underline{\text{all}} \ x \text{ replaced by } y$$

Example

$$\alpha \equiv x + y = s(x) \quad \beta \equiv \forall x \forall y \ x + y = y + x \quad \gamma \equiv \forall x \ x + y = x$$

x_2 is free for x_0 in $\exists x_3(P(x_0, x_3))$,

$f(x_0, x_1)$ is not free for x_0 in $\exists x_1(P(x_0, x_3))$,

x_5 is free for x_1 in $P(x_1, x_3) \rightarrow \exists x_1(Q(x_1, x_2))$.

What Can Go Wrong?

Consider the formula over the type of natural numbers:

$$\forall x \exists y (y > x).$$

This asserts that there is always a larger natural number. Notice that x is free in the formula $\exists y (y > x)$. So if we aren't careful and substitute $x \mapsto y + 1$, then we obtain the statement

$$\exists y (y > y + 1)$$

Which is not true. **This problem arose because y is bound in the formula, so we can't substitute $t = y + 1$ for x .**

Example from Section 7.3 of Logic and Proof

Terms, WFF, and Prop

We have so far distinguished two sorts of syntactically correct strings: terms and WFF. We are interested in *proofs* and *proving theorems*. With the notion of free/bounded variables we can now pick out those WFF which are propositions; $\text{Prop} \subset \text{WFF}$.

We say a WFF is closed if there are no free variables. Propositions are closed WFF.

The distinction between free and bound variables is thus the distinction between propositions and the otherwise syntactically correct WFF.

Universal Elimination

If $\frac{\Sigma}{\forall x \alpha} \mathcal{D}$ is a derivation of $\forall x \alpha[x]$ from hypotheses Σ , where y is free for x in α , then

$$\frac{\frac{\Sigma}{\mathcal{D}} \quad \forall x \alpha[x]}{\alpha[x/y]} \forall E$$

is a derivation of $\alpha[x/y]$ from hypotheses Σ .

Aside: Empty Types?

An interpretation closer to the BHK would have two branches. First we use a proof that any term $x : T$ necessarily has property α and then provide a term $t : T$ to substitute into the predicate.

$$\frac{\begin{array}{c} \Sigma_1 \\ \mathcal{D}_1 \\ \forall x : T, \alpha[x] \end{array} \quad \begin{array}{c} \Sigma_2 \\ \mathcal{D}_2 \\ t : T \end{array}}{\alpha[x/t]} \quad \forall E$$

In practice these are the same *under the assumption that T is non-empty*. Since logicians² do not concern themselves with empty types (models), they (and we) will just use the definition from the previous slide.

²Gentzen, for example, did not consider this in Section 2.21 of Investigations Into Logical Deduction.

Every human is mortal. Sherlock is Human. Therefore, Sherlock is mortal.

Proof of For All

What does it mean to prove something “for all x ”?

A proof of the irrationality of $\sqrt{2}$ starts as follows:

Assume there are some integers A, B such that $\sqrt{2} = A/B$

Eventually deriving a contradiction from this assumption.

The proof itself, in the end, assumes *nothing* of the integers A, B apart from their (possible) existence. Thus it shows that no matter which integers are in place of x, y , the equation

$$(x^2 = 2y^2)$$

can have no integer solutions and hence we may conclude

$$\forall x, y \neg(x^2 = 2y^2)$$

Universal Introduction

If $\frac{\Sigma}{\alpha[x/y]} \mathcal{D}$ is a derivation of $\alpha[x/y]$ from hypotheses Σ , where y does not appear free in Σ nor α , then

$$\frac{\frac{\Sigma}{\mathcal{D}} \alpha[x/y]}{\forall x \alpha} \forall I$$

is a derivation of $\forall x \alpha$ from hypotheses Σ .

Properties of y can't play a role in the proof of $\alpha[x/y]$. They can't appear in the hypotheses Σ of the proof.

Nor can any y be free in α . You should be replacing all instances of y with an x when using the $\forall$ introduction.

Provide a proof to show

$$\forall x(Px \rightarrow Qx), \forall xPx \vdash \forall xQx$$

Provide a proof to show

$$\forall x(Px \rightarrow Qx), \forall x(Qx \rightarrow Rx) \vdash \forall x(Px \rightarrow Rx)$$

Example: When Things Go Wrong

Sherlock is a person. Gladstone is not a person. Therefore, Moriarty is the Queen.

Show $P_s, \neg P_g \vdash Q$

$$\frac{\frac{\frac{P_s}{\forall x P_x} \quad \forall I}{\neg P_g \quad P_g} \quad \forall E}{\perp} \quad MP$$
$$\frac{\perp}{Q} \quad \perp$$

Example: When Things Go Wrong

Sherlock is a person. Gladstone is not a person. Therefore, Moriarty is the Queen.

Show $P_s, \neg P_g \vdash Q$

$$\frac{\frac{\frac{P_s}{\forall x P_x} \quad \forall I}{\neg P_g \quad P_g} \quad \forall E}{\perp} \quad MP$$

$\frac{\perp}{Q} \quad \perp$

From one instance (Sherlock's Personhood) we cannot assume all objects are people.

Example: When Things Go Wrong

Let $\alpha \equiv (x = y)$ and hence $\alpha[x/y] \equiv (y = y)$.

$$\begin{array}{c} \vdots \\ \mathcal{D} \\ \frac{y = y}{\forall x (x = y)} \quad \forall I \end{array}$$

This claims that if something is equal to itself, then everything is equal to that one thing. Obviously not something one would want in their logic!

Note: This is an example of an error resulting from not replacing all y s with x s in the $\forall$ introduction step i.e. not making sure y does not appear free after the substitution.

Existential Introduction

If $\frac{\Sigma}{\alpha[x/y]} \mathcal{D}$ is a derivation of $\alpha[x/y]$ from hypotheses Σ , where y is free for x in α , then

$$\frac{\frac{\Sigma}{\mathcal{D}} \alpha[x/y]}{\exists x \alpha} \exists I$$

is a derivation of $\exists x \alpha$ from hypotheses Σ .

Example(!!!)

Provide a proof to show

$$\forall x P_x \vdash \exists x P_x$$

Provide a proof to show

$$\forall x(Px \rightarrow Qx), \neg Qa \vdash \exists x \neg Px$$

Existential Elimination

Under the BHK interpretation a proof of $\exists x P(x)$ has the form:

$$(t : T, p : P(t)) : (\exists x : T, p(x))$$

If we have as a hypothesis $\exists x P(x)$, then how are we to make use of this in a proof?

Existential Elimination

If $\frac{\Sigma_1}{\exists x \alpha} \mathcal{D}_1$ and $\frac{\Sigma_2}{\alpha[x/y] \rightarrow \gamma} \mathcal{D}_2$ are derivations, and y is neither free in γ nor $\Sigma_1 \cup \Sigma_2$, then

$$\frac{\begin{array}{cc} \Sigma_1 & \Sigma_2 \\ \mathcal{D}_1 & \mathcal{D}_2 \\ \exists x \alpha & \alpha[x/y] \rightarrow \gamma \end{array}}{\gamma} \exists E$$

is a derivation of γ from hypotheses $\Sigma_1 \cup \Sigma_2$.

$$\forall x(Px \rightarrow Qx), \exists xPx \vdash \exists xQx$$

Example: When Things Go Wrong I

Sherlock is a detective. Therefore, everyone is a detective.

$$\exists x D x \vdash \forall x D x$$

$$\frac{\frac{\frac{\overline{Da} \quad 1}{Da \rightarrow Da} \rightarrow I, 1}{\exists x D x} \exists E}{\frac{Da}{\forall x D x} \forall I}$$

Example: When Things Go Wrong II

Sherlock is a human. Gladstone is a dog. Therefore, there exists something that is both a human and a dog.

$$Hs, \exists x Ds \vdash \exists x (Hx \wedge Dx)$$

$$\frac{\frac{\frac{Hs \quad \overline{Ds} \quad 1}{Hs \wedge Ds} \wedge I}{\exists x (Hx \wedge Dx)} \exists I}{\frac{\exists x Ds \quad Ds \rightarrow \exists x (Hx \wedge Dx)}{\exists x (Hx \wedge Dx)} \rightarrow I, 1} \exists E$$

$$\exists xPx \vee \exists xQx \vdash \exists x(Px \vee Qx)$$

Note: Our hypothesis is a disjunction. So we will need the disjunction elimination rule to make use of it.

This breaks the proof into two steps:

① $\exists xPx \rightarrow \exists x(Px \vee Qx)$

② $\exists xQx \rightarrow \exists x(Px \vee Qx)$

Both of these steps will have the same form, so let's focus on the first.

We have reduced the problem to showing:

$$\vdash \exists x P_x \rightarrow \exists x (P_x \vee Q_x)$$

We have reduced the problem to showing:

$$\vdash \exists x P_x \rightarrow \exists x (P_x \vee Q_x)$$

The deduction theorem ($\rightarrow I$) implies this is equivalent to showing:

$$\exists x P_x \vdash \exists x (P_x \vee Q_x)$$

We have reduced the problem to showing:

$$\vdash \exists x Px \rightarrow \exists x (Px \vee Qx)$$

The deduction theorem ($\rightarrow I$) implies this is equivalent to showing:

$$\exists x Px \vdash \exists x (Px \vee Qx)$$

Now our hypothesis has an existential quantifier in it, so we need to use the corresponding elimination rule. Our proof will have a line of the form:

$$\frac{\exists x Px \quad \frac{}{Pa \rightarrow \exists x (Px \vee Qx)} ?}{\exists x (Px \vee Qx)} \exists E$$

How can we complete the proof?

Proof Strategy

We want to show $\vdash \exists x Px \rightarrow \exists x(Px \vee Qx)$ and so far our proof looks like this. What can we do to finish this proof?

$$\frac{\frac{\frac{\overline{Pa} \quad 1}{Pa \vee Qa} \vee I}{\exists x(Px \vee Qx)} \exists I}{\frac{\overline{\exists x Px} \quad 2 \quad Pa \rightarrow \exists x(Px \vee Qx)}{\exists x(Px \vee Qx)} \rightarrow I, 1} \exists E$$

We have proven the first of our subgoals. Go through the same process to prove the second subgoal.

$$\textcircled{1} \exists x Px \rightarrow \exists x (Px \vee Qx)$$

$$\textcircled{2} \exists x Qx \rightarrow \exists x (Px \vee Qx)$$

Use the disjunction elimination to tie these goals together into a proof of the original statement

$$\exists x Px \vee \exists x Qx \vdash \exists x (Px \vee Qx)$$

$$\neg \exists x P_x \vdash \forall x (\neg P_x)$$

Dirk van Dalen, Logic and Structure.

Simon Thompson, Type Theory and Functional Programming.
Chapters 1 - 4.

Jeremy Avigad et al, Logic and Proof³.
Chapters 7, 8, and 9.

³Note: this uses an outdated version of Lean, but is still a good resource for the theory of this course.